

图像压缩编码

By Wei-gang Chen

April 22, 2025

- 这一章的主要内容包括:

- ① DCT 变换
- ② 数字图像压缩编码基础
- ③ 变长编码
- ④ 位平面编码
- ⑤ 游程编码
- ⑥ 变换编码

One dimensional DCT

- 离散余弦变换(Discrete cosine transform, DCT)
 - 1 N.Ahmed 等人在 1974 年提出的正交变换方法, 是一种实变换;
 - 2 DCT 与傅里叶变换紧密相关, all the desirable properties of DFT (such as fast algorithm) are preserved.
 - 3 被认为是对语音和图像信号进行变换的最佳方法;
 - 4 有快速算法, 随着数字信号处理芯片(DSP)的发展, 加上适合于 ASIC, 这就牢固地确立离散余弦变换(DCT)在目前图像编码中的重要地位, 成为 H.264、JPEG、MPEG 等国际编码标准的重要环节.

One dimensional DCT

- DCT 与傅里叶变换的简单对比

特性	傅里叶变换 (FT)	离散余弦变换 (DCT)
基函数	复指数函数 (正弦和余弦)	仅余弦函数
输出类型	复数 (包含幅度和相位信息)	实数 (仅幅度信息)
对称性	对非对称信号处理效果较好	对对称信号处理效果较好
能量集中性	能量分布较分散	能量集中在低频区域, 适合压缩
计算复杂度	较高 (涉及复数运算)	较低 (仅实数运算)

One dimensional DCT

- 一维离散余弦变换: DCT-II(标准DCT)

- ★ 给定 **N-point** 实信号序列 $\{x(m), m = 0, \dots, N-1\}$, 其 **1-D DCT** :

$$X(u) = a(u) \sum_{m=0}^{N-1} x(m) \cos\left(\frac{(2m+1)u\pi}{2N}\right)$$

- ★ 其中 m 是时域索引; u 是频域索引

- ★ $a(u)$ 为归一化系数 $a(u) = \begin{cases} \sqrt{1/N} & u = 0 \\ \sqrt{2/N} & u > 0 \end{cases}$

- ★ 这是最常用的形式, 其它的几种形式可参见维基百科
<http://wapedia.mobi/zh/DCT>

One dimensional DCT

- 前面的 1-D DCT 也可以写成如下形式:

$$X(u) = \sum_{m=0}^{N-1} x(m) c(u, m)$$

$$\star \text{ 其中 } c(u, m) = \begin{cases} \sqrt{\frac{1}{N}} & \text{for } u = 0 \\ \sqrt{\frac{2}{N}} \cos\left(\frac{(2m+1)u\pi}{2N}\right) & \text{for } u = 1, 2, \dots, N-1 \end{cases}$$

- 对于任意的 u , 参与求和运算的各项为:

$$x(0) c(u, 0)$$

$$x(1) c(u, 1)$$

...

$$x(N-1) c(u, N-1)$$

One dimensional DCT

- 续上页, 可得向量点积形式的算式

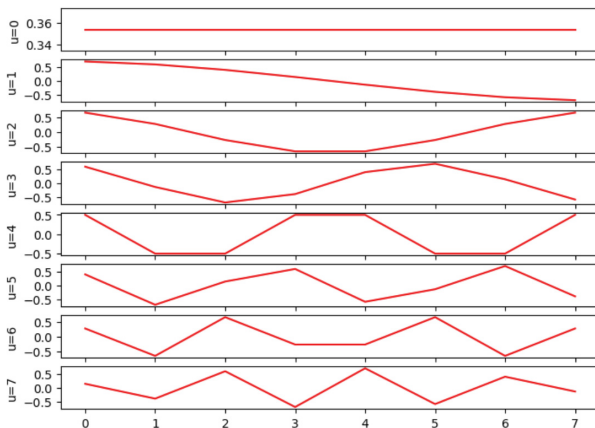
$$X(u) = [c(u, 0), c(u, 1), \dots, c(u, N-1)] \begin{bmatrix} x(0) \\ x(1) \\ \dots \\ x(N-1) \end{bmatrix}$$

- 对所有的 $u = 0, 1, 2, \dots, N-1$, 可以矩阵与向量相乘的形式计算所有的 $X(u)$ 以 $N = 4$ 为例, 矩阵 C 如下

$$C = \begin{bmatrix} 0.5 & 0.5 & 0.5 & 0.5 \\ 0.65 & 0.27 & -0.27 & -0.65 \\ 0.5 & -0.5 & -0.5 & 0.5 \\ 0.27 & -0.65 & 0.65 & -0.27 \end{bmatrix}$$

One dimensional DCT

- 容易发现, the DCT matrix C is orthogonal.
- C 中的每一行, 也称作离散余弦变换的基向量, 下图是 $N = 8$ 的 8 个基向量.



One dimensional DCT

- 1-D DCT 变换的反变换

$$x[m] = \sum_{u=0}^{N-1} a(u) X(u) \cos\left(\frac{(2m+1)u\pi}{2N}\right)$$

- 计算每一个 $x[m]$ 时, u 是自变量.

- 回顾: DFT 的定义

- 给定时间序列, $x[n]$, $n = 1, 2, \dots, N - 1$, 其离散傅立叶变换 (DFT):

$$\text{DFT} \{x[n]\} = X[u] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j2\pi un/N}$$

- 其中: $u = 0, 1, \dots, N - 1$
- 注意欧拉公式: $e^{-jx} = \cos x - j \sin x$

- 结论:

- **DCT 变换可从 Fourier 变换的实部求得, 与 DFT 一样 DCT 存在快速算法!**

2-D DCT

- 2-D 的DCT变换由下式定义:

DCT 变换:

$$C(u, v) = a(u) a(v) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) \cos \left[\frac{(2x+1)u\pi}{2N} \right] \cos \left[\frac{(2y+1)v\pi}{2N} \right]$$

DCT 反变换:

$$f(x, y) = \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} a(u) a(v) C(u, v) \cos \left[\frac{(2x+1)u\pi}{2N} \right] \cos \left[\frac{(2y+1)v\pi}{2N} \right]$$

其中:

$$a(u) = \begin{cases} \sqrt{1/N} & u = 0 \\ \sqrt{2/N} & \text{others} \end{cases}$$

2-D DCT

- 2-D 的 DCT 变换是二维可分离的正交变换: 概念

- 图像变换的一般形式: 正变换

$$T(u, v) = \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) h(x, y, u, v)$$

正向变换核

- 图像逆变换的一般形式

$$f(x, y) = \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} T(u, v) k(x, y, u, v)$$

反向变换核

2-D DCT

- 2-D 的 DCT 变换是二维可分离的正交变换: 可分离变换的概念
 - ★ 如果变换核可写成如下的形式, 则变换是可分离的

$$h(x, y, u, v) = h_1(x, u) h_2(y, v)$$

$$k(x, y, u, v) = k_1(x, u) k_2(y, v)$$

2-D DCT

- 注意: 前面的 DCT 变换式, 即可分离的

$$C(u,v) = a(u)a(v) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x,y) \cos \left[\frac{(2x+1)u\pi}{2N} \right] \cos \left[\frac{(2y+1)v\pi}{2N} \right]$$

- 可写成:

$$C(u,v) = \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x,y) A_{u,x} A_{v,y}$$

其中:

$$A_{u,x} = a(u) \cos \left[\frac{(2x+1)u\pi}{2N} \right]$$

$$A_{v,y} = a(v) \cos \left[\frac{(2y+1)v\pi}{2N} \right]$$

2-D DCT

- 2-D DCT basis images:

- 如果给定频率轴坐标 $u = u_0$ 和 $v = v_0$, 则变换核为

$$h(x, y; u_0, v_0) = A_{u_0, x} A_{v_0, y}$$

- 1 其中

$$A_{u_0, x} = a(u_0) \cos \left[\frac{(2x+1)u_0\pi}{2N} \right], \quad A_{v_0, y} = a(v_0) \cos \left[\frac{(2y+1)v_0\pi}{2N} \right]$$

- 2 显然, 不同的坐标 (x, y) 对应不同的系数 $A_{u_0, x} A_{v_0, y}$, DCT 变换是这些位置的图像值 $f(x, y)$ 与这些系数乘积之和.

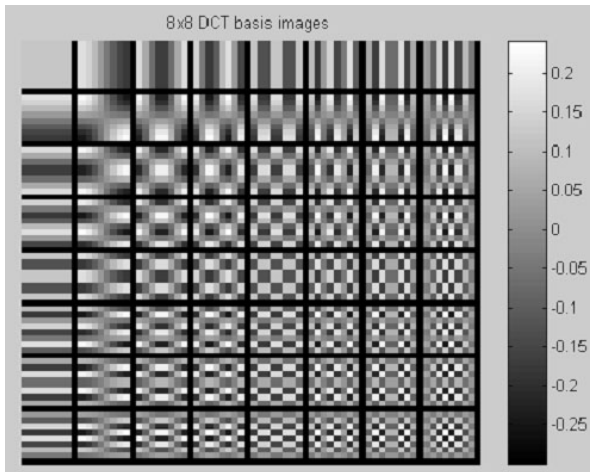
- DCT basis images:

- ① 与前页所论述的 u_0, v_0 一样, 固定 u, v 坐标, 分别计算每个 x, y 位置的系数将得到一个 $N \times N$ 的矩阵, 称作基图像(basis images);
- ② 基图像表示了计算 DCT 变换时每个像素的贡献值. 这些系数 只与坐标有关, 与图像值无关.

2-D DCT

- DCT basis images:

★ 下图是 8×8 的DCT基图像.



• 离散余弦变换的几点说明

- ① DCT变换避免了复数运算。由于图像矩阵是实数矩阵，那么它的DCT也是实数；
- ② DCT是正交变换，其变换矩阵是正交阵，变换核是可分离的。
- ③ DCT有快速算法。
- ④ DCT与IDCT具有相同的变换核，因此具有相同的变换矩阵，即正变换与逆变换可共用同一个算法模块。
- ⑤ DCT具有更强的信息集中能力(**energy compact**)，能将最多的信息放到最少的系数上去。

2-D DCT

- opencv-python 的二维 DCT 变换
 - ★ `dct(x)` 实现对矩阵 `x` 的二维离散余弦变换
 - ★ 下图显示了 `x` 矩阵和变换后的系数矩阵

```
T =  
  
105 112 117 118 119 118 119 120  
106 113 118 118 119 120 120 121  
106 115 121 121 121 121 121 121  
107 116 122 121 121 121 120 120  
108 116 121 119 118 120 120 119  
101 113 121 122 119 121 120 123  
101 113 123 123 121 121 120 123  
102 114 123 125 123 121 120 123
```

```
>> dct (T)  
  
ans =  
  
941.8750 -29.7866 -23.7811 -19.1499 -8.3750 -3.8528 1.0560 -0.5065  
-4.5287 2.4129 5.6060 5.7318 0.6690 0.3931 -0.9657 1.2872  
-2.1068 -3.9238 -2.1705 -1.7277 2.4894 -0.5175 -0.2955 0.1497  
-4.9369 -1.0598 2.0405 -0.1550 -0.3900 0.3950 0.3075 -0.5981  
0.1250 3.5349 0.6649 1.5271 0.3750 0.1442 -0.6813 0.6813  
0.7452 -0.7074 -2.0711 -1.3121 0.2269 -0.3628 0.1241 0.3669  
0.4667 -1.2947 -1.2955 -1.0691 0.4571 -0.0298 -0.5795 -0.6191  
0.0041 1.5801 0.9486 0.6801 -0.5351 0.5372 -0.2276 0.6048
```

2-D DCT

- Open CV 的二维 DCT 变换: `dct()` 和 `idct()`

- `dct()` 用法

```
9  # 变换之前要转换成 DCT 能处理的浮点数
10 src = im.astype(np.float)
11
12 # cv2.dct(src), 离散余弦变换
13 dct_m = cv2.dct(src)
```

- `idct()` 用法

```
18 # 逆 DCT 变换
19 im_idct = cv2.idct(dct_m)
```

2-D DCT

- Open CV 的二维 DCT 变换: `dct()` 和 `idct()` 的例子
 - `dct` 变换后, 将小于阈值的系数置为 0, 作 `idct` 变换, 观察与原图像的差异.

```
9  # 变换之前要转换成 DCT 能处理的浮点数
10 src = im.astype(np.float)
11
12 # cv2.dct(src), 离散余弦变换
13 dct_m = cv2.dct(src)
14
15 # 数据压缩, 只保留 64 * 64 范围内的 dct 系数
16 resi_dct = np.zeros(im.shape)
17 resi_dct[0:64, 0:64] = dct_m[0:64, 0:64]
18
19 dct_resi_log = np.log(abs(resi_dct))
20
21 # 逆 DCT 变换
22 im_idct = cv2.idct(resi_dct)
23
24 # 显示比较
25 plt.subplot(131)
26 plt.imshow(im)
27 plt.title('original image')
28 plt.subplot(132)
29 plt.imshow(dct_resi_log)
```

压缩编码基础：图像数据压缩的可能性

- 图像信号可以压缩的根据有两个方面

- ① 图像信号存在大量的冗余。

- 信息是由数据来表示的。同样的信息，不同的表示方法使用的数据量不同，使用较多数据量的方法中，有些数据是冗余的。

- ② 可以利用人的视觉特性。

- 在不被人眼主观察觉的前提下，减少表示信号的精度，以一定的失真换取数据量的减少。

压缩编码基础：图像数据压缩的可能性

- 图像压缩：减少表示图像的数据量

- ① 在图像传输的时候，可传输较少的数据量，但不减少图像所表达的信息量；
- ② 在图像存储时，可减少所需的存储空间，同时减少存储所需的时间。

- Compression rate

- 设 n_1 和 n_2 是两种不同的表示方法表示同一个信息所需的数据量，则压缩率为：

$$C_R = \frac{n_1}{n_2}$$

压缩编码基础：图像数据压缩的可能性

- 图像信号的冗余度表现在结构和统计两个方面
 - ★ 图像信号结构上的冗余度表现为：图像具有很强的空间(帧内)和时间(帧间)相关性。
 - ★ 以电视信号为例，电视画面的大部分区域变化缓慢，即空间相关性；连续若干个画面变化不大，特别是背景部分。即时间相关性。

压缩编码基础：图像数据压缩的可能性

- Coding redundancy(编码冗余):

- ① 符号(Symbol): 表示信息的编码(codes that human represent information).
- ② 具有较高出现频度的符号, 应该赋予一个具有较短长度的码(codes of less amount of data);
 - ★ 即: 对一个出现频度高的灰度赋予较少比特的编码, 出现频度较小的灰度赋予较多比特的编码, 能实现数据压缩;
- ③ 这种思路即 变长编码(variable-length coding).

压缩编码基础：图像数据压缩的可能性

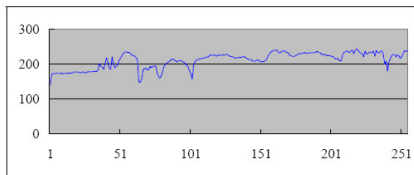
- Inter-pixel redundancy(像素间冗余)

- ① 图像的邻域像素往往存在很高的相关性;
- ② 表现形式为: 邻域范围内的像素, 往往具有相似的灰度值.

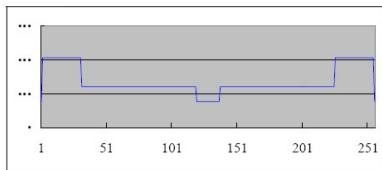
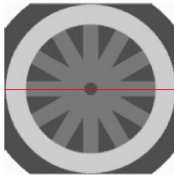
压缩编码基础：图像数据压缩的可能性

- Inter-pixel redundancy(像素间冗余)

- 下图是像素间冗余的一个示意图：



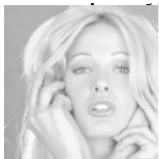
Pixels of line 128 of *wheel*:



压缩编码基础：图像数据压缩的可能性

- Psycho-visual redundancy(心理-视觉冗余)

- ① 对于人类的视觉感知而言，某些信息相对另一些信息而言不那么重要。
 - ② 对这样的灰度信息作适当的量化，不会影响到图像的视觉质量。
- 下图是这类冗余的一个示意图：



8 bits/pixel



5 bits/pixel

压缩编码基础：图像数据压缩的可能性

- Inter-frame redundancy(帧间冗余)

- ★ 相邻帧存在很强的相似性;
- ★ 这类冗余可参见 `mobile.cif`:



前言：图像质量的主观评价和客观评价

- 人眼是图像或视频通信系统的信宿, 所以图像质量的评测直接或间接都必须依赖于主观评价.
- 图像质量的主观评价结果和许多因素有关, 包括
 - ① 评价人员的经验;
 - ② 所选用的图像;
 - ③ 观看的条件: 如环境照明, 显示器对比度, 最高亮度, 观看距离等.

前言：图像质量的主观评价和客观评价

- 关于客观评价:

- ① 图像质量的主观评价不能由客观评价完全代替, 但在论文, 报告等学术交流中需要定量的客观评价;
- ② 图像质量的客观评价有均方误差, 信噪比 **SNR**, 峰值信噪比 **PSNR**等.

- 均方误差(Mean Squared Error (MSE))

$$MSE = \frac{1}{MN} \sum_{i=1}^M \sum_{j=1}^N \left(S(i, j) - \hat{S}(i, j) \right)^2$$

- S 为原始图像, $\hat{S}$ 为编码传输后的解码图像.

前言：图像质量的主观评价和客观评价

- 信噪比 SNR

$$SNR = 10 \times \log_{10} \frac{\sigma_s^2}{MSE}$$

- 其中 $\sigma_s^2 = \frac{1}{MN} \sum_{i=1}^M \sum_{j=1}^N [S(i, j)]^2$, 是原始图像的平均功率.

- 峰值信噪比 PSNR(Peak Signal-to-Noise Ratio)

$$PSNR = 10 \times \log_{10} \frac{(\max(I))^2}{MSE}$$

- 如果图像被均匀量化为 256 个 level, 则 $\max(I) = 255$.
- **Typical values for the PSNR in lossy image and video compression are between 30 and 50 dB, where higher is better.**

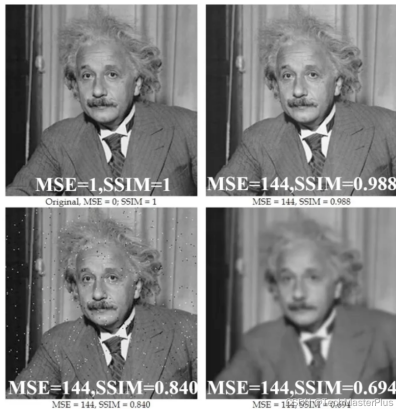
前言：图像质量的主观评价和客观评价

- 客观评价指标的优缺点:

- ① 采用均方误差或信噪比作为图像质量的评价依据, 其优点是容易计算, 数学上也容易处理.
- ② 但均方误差等评价与图像质量的主观评价会存在偏差.
 - 原因: 均方误差是对图像重建误差在全图像范围的平均, 不反映图像局部的质量; 人眼往往对一些局部比较敏感, 而对另一些不敏感.

前言：图像质量的主观评价和客观评价

- 结构相似性度量(Structural Similarity Index Measure) – SSIM
 - 下图示意了 **MSE(PSNR)** 相近但图像质量相差甚远的例子, 也背书了 **SSIM** 评价指标的价值.



前言：图像质量的主观评价和客观评价

- SSIM 基于样本之间的三个比较：亮度、对比度和结构

- 亮度：以平均灰度衡量。

- ① 计算样本均值

$$\mu_x = \frac{1}{H \times W} \sum_i \sum_j x(i, j), \quad \mu_y = \frac{1}{H \times W} \sum_i \sum_j y(i, j)$$

- ② 计算亮度比较函数

$$\ell(x, y) = \frac{2\mu_x\mu_y + C_1}{\mu_x^2 + \mu_y^2 + C_1}$$

- ③ Note: 当 $\mu_x = \mu_y$, ℓ 取最大值 1, C_1 用于避免分母为 0。

前言：图像质量的主观评价和客观评价

- SSIM 基于样本之间的三个比较：亮度、对比度和结构

- 对比度：通过灰度标准差衡量。

- ① 计算样本标准差的无偏估计 σ_x (类似可计算 σ_y) :

$$\sigma_x = \left[\frac{1}{H \times W - 1} \sum_{i=1}^H \sum_{j=1}^W (x(i, j) - \mu_x)^2 \right]^{1/2}$$

- ② 计算对比度比较函数

$$c(x, y) = \frac{2\sigma_x\sigma_y + C_2}{\sigma_x^2 + \sigma_y^2 + C_2}$$

- ③ Note: 当 $\sigma_x == \sigma_y$, c 取最大值 1, C_2 用于避免分母为 0。

前言：图像质量的主观评价和客观评价

- SSIM 基于样本之间的三个比较：亮度、对比度和结构
 - 结构：使用相关性系数衡量。

① 计算协方差

$$\sigma_{xy} = \frac{1}{H \times W - 1} \sum_i \sum_j (x(i, j) - \mu_x)(y(i, j) - \mu_y)$$

② 计算结构比较函数

$$s(x, y) = \frac{\sigma_{xy} + C_3}{\sigma_x \sigma_y + C_3}$$

前言：图像质量的主观评价和客观评价

- SSIM 基于样本之间的三个比较：亮度、对比度和结构

- ① 计算结构相似性度量

$$\text{SSIM} = \ell(x, y)^\alpha \cdot c(x, y)^\beta \cdot s(x, y)^\gamma$$

- ② 当取 $\alpha = \beta = \gamma = 1$, $C_3 = C_2/2$,

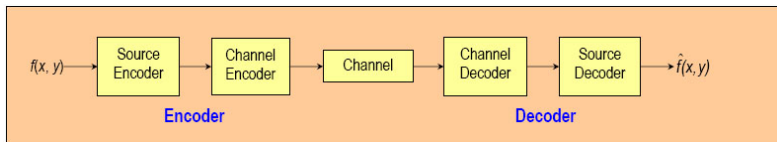
$$\text{SSIM} = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}$$

前言：图像质量的主观评价和客观评价

- SSIM 基于样本之间的三个比较：亮度、对比度和结构
 - 通常以滑窗的形式计算局部 **SSIM**，然后求整体的均值。
 - ① 图像局部的均值和方差往往变化剧烈；
 - ② 图像上不同区块的失真程度可能差异显著；
 - ③ 类比人眼每次只能聚焦于一处的特点，可以一定的步长计算两幅图各个对应 **sliding window** 下子图像的 **SSIM**，然后取平均值作为整体的 **SSIM**，称为 **Mean SSIM**。

图像压缩模型

- 一个压缩系统包含两个不同的结构模块: an encoder and a decoder.
- The figure shows a general compression system model.



- 关于前页的压缩系统模型.

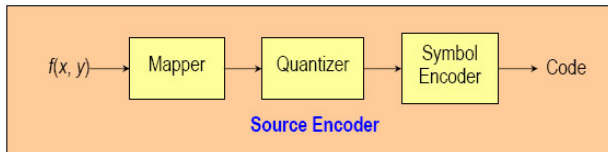
- ① 编码器(**Encoder**): 将输入数据转换成适合传输的信号. 编码器根据输入数据产生一个符号序列;
 - **Source encoder**: 减少或消除各种冗余.
 - **Channel encoder**: 在传输的数据中加入各种控制信息, 以对抗噪声干扰, 如 **CRC** 码, 海明码.
- ② **Channel(信道)**: media for storage or transmission.
- ③ **Decoder**: 将接收到的信号转换成输出数据.
 - **Channel decoder**: error detection and correction.
 - **Source decoder**: 恢复成原始数据.

图像压缩模型

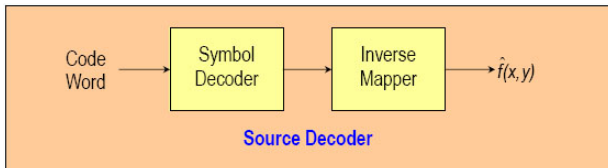
- 信源编码器和信源解码器

- 变换数据以减少冗余 (coding, interpixel, and psycho-visual).

- Procedure of source encoder



- Procedure of source decoder



- 信源编码器和信源解码器

- ★ Mapper(映射): 输入数据进行某种形式的转换以减少像素间冗余(通常转换后的数据是非可视化的).
- ★ 这种操作往往是可逆的(reversible operation), e.g., run-length coding(行程编码), DCT 等.

- 信源编码器和信源解码器

- ★ Quantizer(量化): 在符合一些保真度准则的前提下, 减少 Mapper 步骤输出数据的精度 ;
- ★ 通常这个过程是不可逆的. 这个过程能减少心理-视觉冗余(the psychovisual redundancies).

- 信源编码器和信源解码器

- ★ 符号编码器(Symbol coder): 产生定长或不定长的编码 (fixed or variable - length code), 来表示量化器的输出.
- ★ e.g., 变长编码通过将最短的码赋给最频繁出现的输出值来减少编码冗余.

- 信道编码器和信源解码器

- ★ 通过在源编码器的输出编码数据中加入冗余的控制信息, 来避免和减少信道噪声的影响;
- ★ 如 CRC 码, 海明码等.
- ① **Channel encoding:** 加入比特, 以增加码字(code words)之间的距离.
- ② **Channel decoding:** 解码恢复出原始的编码(即信源编码器的输出).

- 信息量

- ① 客观事物的状态及其变换的表征和反映称作 **信息**.
- ② 有用的信息应该能排除不确定性, 排除的不确定度越多则信息量越大;
- ③ 在概率论中, 不确定度用随机事件发生的概率来描述.
 - 因此, **事件所蕴含的信息量可以用与随机事件发生的概率有关的某个量来度量.**

- 信息量: Shannon 定义的度量标准

- ★ **Shannon** 以事件出现的概率的对数来度量信息量.
- ★ 如果某个事件 x 出现的概率为 $p(x)$, 则此事件提供的信息量为:

$$I(x) = \log \frac{1}{p(x)} = -\log p(x)$$

- 信息量: 结论

- ★ 对单个事件而言, 小概率事件信息量大.

• 信源

- ① 能够产生信息的事物称作 **信源**.
- ② 若信源 X 可能产生的信息是 $x_1, x_2, \dots, x_n$, 它们出现的概率分别为 p_i , 则该信源可表示成

$$X = \left\{ \begin{array}{cccc} x_1 & x_2 & \dots & x_n \\ p_1 & p_2 & \dots & p_n \end{array} \right\}$$

- 信源: 平均信息量

- ① 信源的平均信息量: 定义

$$H(X) = \sum_{i=1}^n p_i I(x_i) = - \sum_{i=1}^n p_i \log(p_i)$$

- ② $H(X)$ 称作信源 X 的熵.

- 信源的熵

- ① $H(X)$ 总是非负的, 当所有的事件出现的概率相等时, 熵取最大值.
- ② 若以 2 为底, 则熵的单位是 “比特 / 符号”
 - 可理解成信源发出一个符号所携带的平均信息量.

图像的信息量和信息熵

- 经过采样、量化过程的数字图像信源是离散信源.
 - ① 离散信源支配着一个由 K 个符号 $\{a_1, a_2, \dots, a_K\}$ 组成的符号集.
 - 如果以 8 比特量化, 则有 256 个符号.
 - ② 各个符号出现的概率为 $p(a_i)$, 且 $\sum_{i=1}^K p(a_i) = 1$.

信息量和信息熵

- 从理论上讲, 信息熵是表示一个符号所需的最少的比特数.

★ 下表给出了两个例子

Symbol	Probability	$H(S) = -\sum (1/4) \times \log_2(1/4) = 2$ → In average, minimum 2 bits required
A	1/4	
B	1/4	
C	1/4	
D	1/4	

Symbol	Probability	$H(S) = -[(1/2) \times \log_2(1/2) + 2 \times (1/8) \times \log_2(1/8) + (1/4) \times \log_2(1/4)] = 1.75$ → In average, minimum 1.75 bits required
A	1/2	
B	1/8	
C	1/8	
D	1/4	

熵编码的一些基本概念

- 熵编码的一些基本概念: 编码
 - 设信源

$$X = \left\{ \begin{array}{cccc} x_1 & x_2 & \dots & x_n \\ p_1 & p_2 & \dots & p_n \end{array} \right\}$$

- ① 编码器的作用就是将信源消息集 X 变成码字集 $W = \{w_1, w_2, \dots, w_n\}$.
- ② w_i 称作 x_i 的代码.
- ③ **编码** 是由信源消息集到码字集的一种映射.

熵编码的一些基本概念

• 熵编码的一些基本概念: 编码

- ① 码字由一些基本符号构成, 所有的基本符号构成符号集

$$A = \{a_1, a_2, \dots, a_n\}$$

- 对图像编码而言, 符号集为 $\{0, 1\}$.

- ② w_i 中的基本符号个数称作码字 w_i 的 **码长**.

- ③ 如果每个消息(如图像中的每个像素值)的码长都相等, 则为**等长码**, 否则为 **不等长码**.

- ④ 前页信源所有消息的平均码长为:

$$\bar{L} = \sum_{i=1}^n p_i L_i$$

熵编码的一些基本概念

- 香农无失真编码定理和编码效率

① **定理**: 在无干扰的条件下, 无失真编码的平均码长存在一个下限, 该下限就是原始图像的熵.

② **编码效率**:

$$\eta = \frac{H(X)}{\bar{L}}$$

③ **压缩比**: 原始图像的平均比特率为 n , 压缩编码后为 n_d , 则压缩比

$$C = \frac{n}{n_d}$$

④ 无失真编码可达到的最大压缩比

$$C = \frac{n}{H(x)}$$

几种变长编码：哈夫曼编码

- ASCII 编码等固定长度编码方案, 所有的字符都使用相同的比特数目.
- 如果所有字符的使用频率都相等, 则固定长度编码是空间效率最高的编码方案.
- 在文档中, 有些字符出现的频率比其它的字符出现的频率高. 如 e, s 或 t 比 x, z, j 等出现的频率高.
- 频率相关编码的基本思路是让出现频率高的字符编码较短(较少的比特), 以利于减小码率.

① 哈夫曼编码

② 算术压缩

几种变长编码：哈夫曼编码

- 哈夫曼编码的基本思路

- 在文档中，往往一些字符比其它的字符更多地出现，让这些字符用较短的编码。其代价是 其它字符使用比固定长度编码更长的代码来表示。

- 一个例子：假设传送的电文是‘ABACCD A’。四种字符，‘A’出现三次，‘B’和‘D’各 1次，‘C’ 2次。

- 以固定长度编码，每个字符 2 比特，共 14 比特。
- 假设‘A’的编码为 1 比特，‘C’的编码为 2 比特，‘B’和‘D’ 分别为 3 比特，则总长度为 13 比特。
- 问题是如何设计这样的编码？且字符是可区分、不会产生错误的解释。

几种变长编码：哈夫曼编码

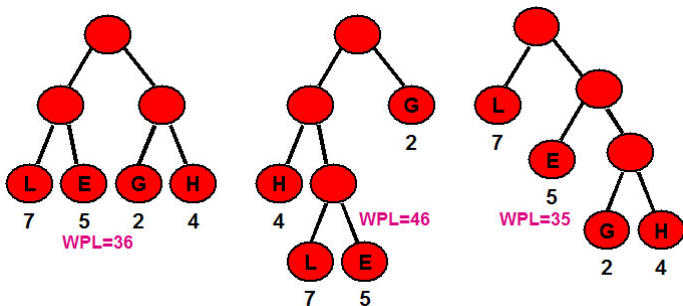
- 哈夫曼编码从 Huffman 树(Huffman tree)得到.
- 路径长度:
 - 从树中的一个结点到另一个结点的分支构成两个结点之间的路径;
 - 路径上分支的数目称作路径长度.
 - 从根到每一结点的路径长度之和为树的路径长度.

几种变长编码：哈夫曼编码

- 引入权的概念, 结点的带权路径长度是该结点到树根之间的路径长度与结点上权的乘积.
- 树的带权路径长度: 树中的所有叶子结点的带权路径长度之和.

几种变长编码：哈夫曼编码

- 下图是树的带权路径长度计算的一个例子:



- 以中间的二叉树为例,

$$WPL = 4 \times 2 + 7 \times 3 + 5 \times 3 + 2 = 46$$

几种变长编码：哈夫曼编码

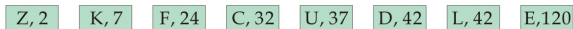
- 如何构造 Huffman 树? Huffman 最早给出了一个带有一般规律的算法, 即 Huffman 算法.
 - 假设给定 n 个结点, 它们的权值分别为 $w_1, w_2, \dots, w_n$.
 - ① 构造 n 棵二叉树的集合 $F = \{T_1, T_2, \dots, T_n\}$, 每棵二叉树只有一个结点(根结点), 其权值为 w_i . 左右子树均空.
 - ② 在 F 中选取两棵根结点的权值最小的树作为左右子树构造一棵新的二叉树, 置新的二叉树的权值为 它的左右子树的权值之和.
 - ③ 在 F 中删除上述构成子树的二叉树, 且将新构造的二叉树加入到 F 中.
 - ④ 重复步骤 2 和 3, 直到 F 中只包含一棵二叉树为止. 这棵树即 Huffman 树.

几种变长编码：哈夫曼编码

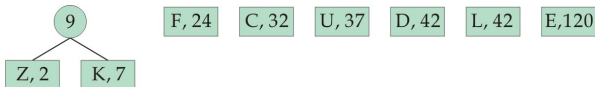
- 假设文本中出现 8 个字符, 如下图所示:

字母	C	D	E	F	K	L	U	Z
频率	32	42	120	24	7	42	37	2

- 步骤1, 构造 8 棵二叉树的集合



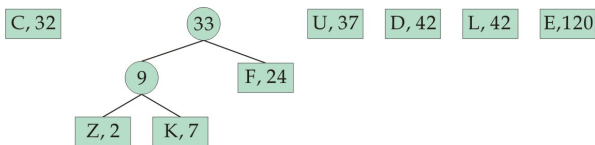
- 步骤2, 上述二叉树集合中权值最小的两棵二叉树作为左右子树构造一棵新的二叉树. 且将它加入到树的集合.



频率相关编码：哈夫曼编码的例子

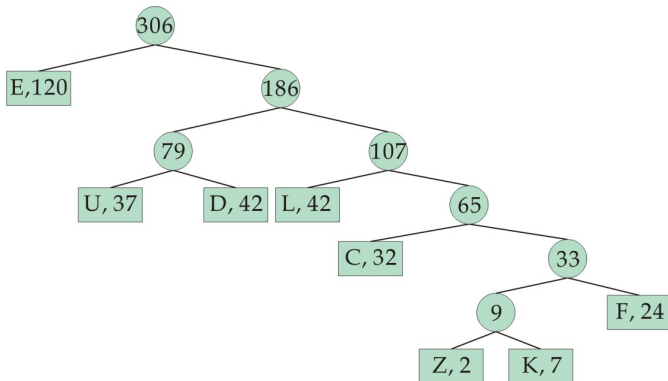
- 前页例子(续)

- 重复前页步骤 2,



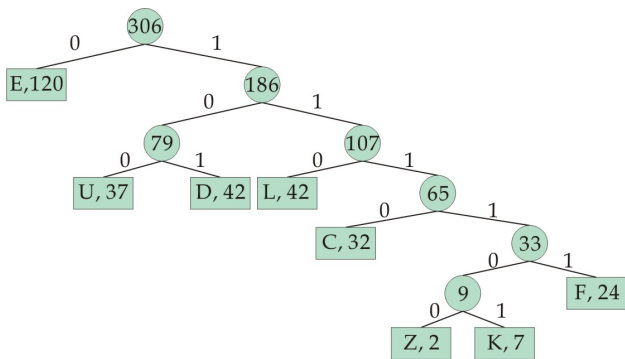
频率相关编码：哈夫曼编码的例子

- 前页例子的最后结果



频率相关编码：哈夫曼编码的例子

- Huffman 树构造完成后, 从根结点开始, 将 '0' 标记于对应左子树的那条边上, '1' 标记于对应右子树的那条边上. 则前页例子如下所示



频率相关编码：哈夫曼编码的例子

- 从根结点开始到每个叶子结点的路径上的‘0’或‘1’比特串即为字符的编码.

字符	C	D	E	F	K	L	U	Z
频数	32	42	120	24	7	42	37	2
编码	1110	101	0	11111	111101	110	100	111100

频率相关编码：哈夫曼编码

- 译码同样也需要编码过程所建立的 Huffman 树.
- 译码的过程是对编码比特串进行如下处理: 从根出发, 按'0'向左子树, '1'向右子树, 直至树叶结点.
 - 如有比特串: 1011001110111101, 利用前页的 Huffman 编码树, 得到结果:

DUCK

频率相关编码：哈夫曼其人

- 成就: Huffman 编码

- 生平

- 1925 年出生. 1944 年毕业于 Ohio 州立大学, 后加入美国海军. 兵役结束后, 回到 Ohio 州立大学继续学业. 1949 获硕士学位, 1953 年于 MIT 获博士学位. 在 MIT 期间以论文的形式发表了 Huffman 编码的成果.
- 1953 年毕业后留 MIT 任教, 1962 年升为教授. 1967 年去加州大学 Santa Cruz 分校. 1994 年退休.
- 1999 年因癌症去世.

- 荣誉

- IEEE Fellow.
- 获得过 Hamming 奖章, Louis E. Levy 奖章. 1998 年获 IEEE 因技术创新而设的 Jubilee 金奖....

Huffman 编码的优缺点

- Huffman 编码的优点是能很好地与待编码的符号的出现频率相匹配, 使平均码长达到最短.
- 缺点是实现复杂
 - 当需要编码的符号数目很多, 构造 Huffman 的运算量会很大.
 - 如果有 K 个符号, 一般需要 $K - 2$ 次信源符号合并, $K - 2$ 次安排代码.

- 关于算术码(arithmetic coding)

- ① 在算术码中, 信源符号和码字之间没有一一对应关系;
- ② 输入的符号序列被映射为实数轴 $[0, 1)$ 区间内的某一个子区间;
- ③ 随着输入符号的增加, 实数子区间不断减小;
- ④ 子区间减少, 将导致表示此区间所需的比特数变得更多.
- ⑤ 在接收端, 由这个实数可以唯一地解码成一个字符串.

频率相关编码：算术编码

- 被编码的字符需要有一个出现频率, 还需要分配一个数字范围(概率大的有较大的区间, 概率小的有较小的区间).

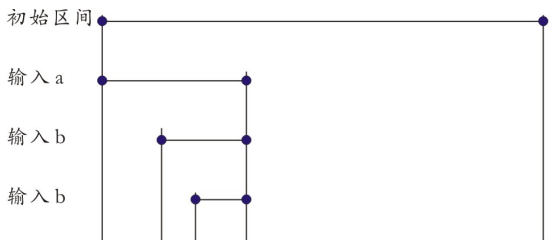
★ 假设信源包含两个符号: a , b . 下表给出了这样的表格

符号	出现频率 (%)	子区间
a	30%	[0, 0.3)
b	70%	[0.3, 1)

★ 假设输入符号串为 'abb', 下页给出算术编码的基本思路.

频率相关编码：算术编码

- 一个简单的例子, 实数区间随着符号输入而减小



- ① 输入符号 'a', 区间由 $[0, 1) \rightarrow [0, 0.3)$;
- ② 输入符号 'b', 区间由 $[0, 0.3) \rightarrow [?, 0.3)$;
- ③ 输入符号 'b', 区间由 $[?, 0.3) \rightarrow [??, 0.3)$;

- 定义算术编码的算法中使用的符号

- 1 **range**: 编码输出数值的落入范围;
- 2 **high**: **range** 的上界;
- 3 **low**: **range** 的下界;
- 4 **high_range(c)**: 函数, 求符号 c 的区间上界(见前页表);
- 5 **low_range(c)**: 函数, 求符号 c 的区间下界;

- 有了前页的表格, 算术编码的算法可描述如下:

set low to 0.0, set high to 1.0.

While there are still input symbols do

get an input symbol

range = high - low

high = low + range*high_range(symbol)

low = low + range*low_range(symbol)

End of While

output low

频率相关编码：算术编码

- 一个例子：符号、出现的概率、分配的区间

基于频率给字符分配区间		
字符	频率(%)	子区间
A	25	[0, 0.25)
B	15	[0.25, 0.4)
C	10	[0.4, 0.5)
D	20	[0.5, 0.7)
E	30	[0.7, 1.0]

频率相关编码：算术编码

- 一个例子：待编码的字符串为 'ABADCAD'，则编码过程如下所示：

注意编码的三个步骤：

$$\text{range} = \text{high} - \text{low}$$

$$\text{high} = \text{low} + \text{range} * \text{high_range}(\text{symbol})$$

$$\text{low} = \text{low} + \text{range} * \text{low_range}(\text{symbol})$$

算术编码过程示意		
新字符	Low Value	High Value
	0	1
A	0	0.25
B	0.0625	0.1
A	0.0625	0.071875
D	0.0671875	0.0690625
C	0.0679375	0.068125
A	0.0679375	0.067984375
D	0.067960938	0.067970313

- 算术编码的解码算法可描述如下:

Get encoded number

Do

find symbol whose range straddles(跨越) the number;

output the symbol;

range = High_value(symbol) - Low_value(symbol);

subtract low_value(symbol) from encoded number;

divide encoded number by range;

End of While

频率相关编码：算术编码

- 假设字符频率和分配的区间如前页表, 对编码实数的解码过程如下图所示

注意解码的三个步骤:

range = High_value(symbol) - Low_value(symbol);

subtract low_value(symbol) from encoded number;

divide encoded number by range;

算术编码解码过程示意			
Encoded Number	Output Symbol	Low Value	High Value
0.067960938	A	0	0.25
0.2718	B	0.25	0.4
0.1456	A	0	0.25
0.5825	D	0.5	0.7
0.4125	C	0.4	0.5
0.1250	A	0	0.25
0.5	D	0.5	0.7
0			

- 教材的例子, 例 7.3.5

- ★ 四信源符号, 五符号输入序列“ $a_1a_2a_2a_3a_4$ ”, 要求写出算术编码
- ★ **Step 1**, 以表格的形式写出各个符号的分配区间.

: 给符号分配区间

符号	频率(%)	子区间
a_1	20	$[0, 0.2)$
a_2	40	$[0.2, 0.6)$
a_3	20	$[0.6, 0.8)$
a_4	20	$[0.8, 1]$

频率相关编码：算术编码

- 教材的例子, 例 7.3.5

- ★ 四信源符号, 五符号输入序列“ $a_1a_2a_2a_3a_4$ ”, 要求写出算术编码
- ★ Step 2, 算术编码过程.

: 编码过程

符号	Low Value	High Value
	0	1
a_1	0	0.2
a_2	0.04	0.12
a_2	0.056	0.088
a_3	0.0752	0.0816
a_4	0.08032	0.0816

- CABAC – Context Adaptive Binary Arithmetic Coding

- ★ H.264, HEVC 所采用的熵编码方案, 与H.264的另一种熵编码方案 CAVLC(Context Adaptive VariableLength Coding)相比, 对于PSNR = 30 - 38 dB的视频质量, 平均可节约9%到14%的码流.

- ★ CABAC 编码过程包括:

- ① 待编码语法元素二进制化;
- ② 上下文建模(确定上下文索引);
- ③ 算术编码.

- CABAC: 待编码语法元素二进制化
 - CABAC 使用二值(0和1)算术编码, 非二进制符号在编码前先转换成二进制数;
 - 可采用的二值化方案: 一元码(Unary Binarization), 截断一元码, k 阶指数哥伦布编码等.

- CABAC: 上下文建模

- ① CABAC 将输入的符号区分为 MPS (高概率符号)和LPS(低概率符号), 例如: 若 1 为 MPS, 则 0 为 LPS;
- ② 将概率值区间 $[0.01875, 0.5]$ 划分成 64 个量化值, 这些概率对应于 LPS 符号, 且对应 0 – 63 的一个索引;
- ③ 若当前 LPS 的概率为 P_{LPS} , 则 MPS 的概率为 $1 - P_{LPS}$;

• CABAC: 上下文建模

- 编码过程中, 上述概率是动态变化的:
- 当出现一个 **MPS** 或 **LPS** 符号, 则概率值相应地发生变化, 具体按以下公式

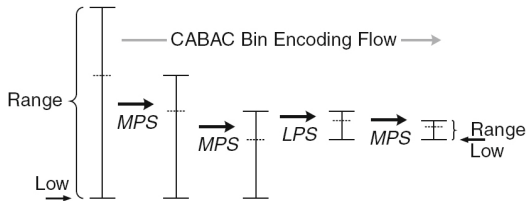
$$p_{new} = \begin{cases} \max(\alpha \cdot p_{old}, p_{62}), & \text{if } bin = MPS \\ 1 - \alpha \cdot (1 - p_{old}), & \text{if } bin = LPS \end{cases}$$

- 概率模型的变化以查表的形式完成.

频率相关编码: CABAC编码

- CABAC: 二进制算术编码

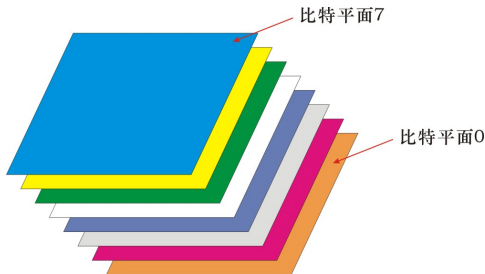
- 使用两个变量: **Low(10bits)**, **Range(9bits)**
- 出现一个 **MPS** 和 **LPS** 符号时, 两个变量的变化如下图所示
 - 当 **Range** 小于 **0x100**, 编码器输出一个比特.



位平面编码：位平面

• 位平面

- ① 一幅用 m 个比特表示其灰度值的图像，可以看作 m 个二值图像序列；
- ② 以 8 比特灰度图像为例，看作 8 个二值图像。某个像素值，第 0 个比特为 1，则第 0 个二值图像的对应像素为 1；否则为 0。
- ③ 以下是一个位平面的示意图：



位平面编码：位平面编码

- 位平面分解后得到的是二值图像, 像素值只有 0 和 1 两种.
- 在位平面图中会存在大量的具有相同值的区域.
- 以下是 Lenna 的位平面图 7(BitPlane.m):



位平面编码：位平面编码

- 由于位平面图中会存在大量的具有相同值的区域, 一种简单的而有效的压缩方法是将位平面图像分成块, 对全 0 和全 1 的块表示成码 0 和 11.

0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

码字0

1	1	1	1
1	1	1	1
1	1	1	1
1	1	1	1

码字11

- 对出现频率高的块赋予较短的码字。
- 对 0 和 1 混杂的块以 10 作为编码前缀, 后面跟比特模式(即不作压缩).

位平面编码：位平面编码

- 另一种对位平面图像编码的方法是行程编码.

- 游程编码, 也叫行程编码, 是一种比较简单的编码方式.
 - ① 与 **Huffman** 编码和算术编码的差别: 不试图从统计每个符号出现的频率出发, 实现减少每个符号的比特数目的目的
 - ② 行程编码分析待发送的符号串, 以一个符号以及该符号连续出现的次数来代替符号串进行发送.
 - ③ 符号连续出现的次数(**a single symbol and a count**)称作行程.
 - 如: “aaaabbbaaa” → “4a2b3a”

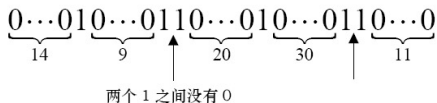
游程编码：适用于大量连续的比特‘0’或‘1’的情形

- 行程编码的适用场合:

- 如传真等应用中, 传输的图像会有很多连续个 ‘0’ (对应白色点). 可以只传送每个‘0’序列的长度, 而在两个‘0’序列之间 默认出现一个比特‘1’.

游程编码：适用于大量连续的比特‘0’或‘1’的情形

- 以下以一个例子来说明这种编码方案的原理

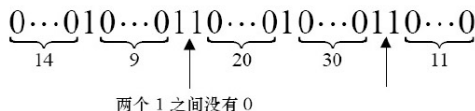


- 假设用四比特的无符号数表示 0 序列的长度，默认两个 0 序列之间是一个 1.
- 第一个‘0’序列长度为14，编码为 1110(十进制数 14)，第二个‘0’序列长度为9，编码为1001.
- 接下来有两个连续的‘1’，应该如何表示？

行程编码：适用于大量连续的比特‘0’或‘1’的情形

- 行程编码的例子(续)

- 连续的‘1’，可以认为中间有序列长度为 0 的比特‘0’.



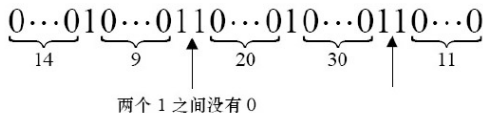
序列长度(编码): 1110 1001 0000

- 上图是此时的编码结果，接下来有 20 个‘0’。超过了四比特无符号数能表示的最大值！

行程编码：适用于大量连续的比特‘0’或‘1’的情形

● 行程编码的例子(续)

- 序列中有 20 个连续的‘0’。超过了四比特无符号数能表示的最大值。
- 所采用的办法是用连续的四比特来表示这个长度。若一个编码的四比特编码是全‘1’，则其后的四比特与它一起 对应同一个序列，直到一个非全‘1’的组合为止。
- 下图是完整的编码结果，注意长度为30时的处理方式。



序列长度(编码): 1110 1001 0000 1111 0101 1111 1111 0000
0000 1011

行程编码：适用于大量连续的比特‘0’或‘1’的情形

- 行程编码的例子(续)

- 注意：前面介绍的方法适用于有大量连续的‘0’或‘1’，若‘0’和‘1’频繁交替出现，不仅不能减少数据量，反而有可能产生一个更大的比特流。

- 变换编码的基本思想

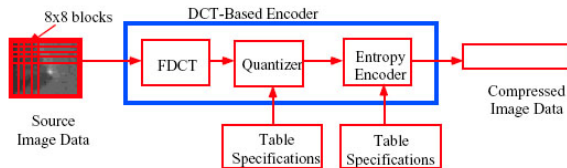
- 以某种可逆的正交变换把给定的图像变换到频率域, 利用频率域数据的特点, 用一组非相关的变换系数来表示原图像, 将尽可能多的信息集中到尽可能少的变换系数上, 使多数系数只携带尽可能少的信息, 实现用较少的数据表示较大的图像数据信息, 从而达到压缩数据的目的.

• 变换编码的一般步骤

- ① 将待编码的图像分解成大小为 $n \times n$ 的子图像, 通常取 $n = 4, 8, 16, 32$.
- ② 对每个子图像进行正交变换(如DCT变换), 得到子图像的变换系数, 其实质是把空间域表示的图像转换成频率域表示的图像.
- ③ 对变换系数进行量化.
- ④ 使用霍夫曼变长编码或游程编码等无损编码器对量化后的系数进行编码, 得到压缩后的图像数据.

Jpeg 图像编码过程

• JPEG 编码框图

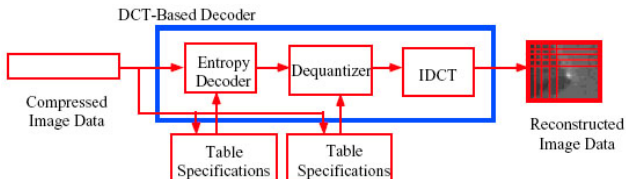


Reference: DIS 10918-1 Document

- 1 **JPEG** 将源图像划分成 8×8 的块, 每块分别进行 **DCT** 变换. 通过 **DCT** 变换消除像素在空间域 的相关性;
- 2 根据给定的量化表, 对 **DCT** 系数进行量化. 量化是通过降低 **DCT** 系数的精度来进一步实现数据压缩.
- 3 按熵编码表进行熵编码, 进一步消除图像数据的统计相关性.

The JPEG Baseline Algorithm

- JPEG 解码框图



Reference: DIS 10918-1 Document

变换编码：变换的选择

- 变换编码常用的变换包括:

- ① DCT 变换.
- ② 沃尔什-哈达玛变换.
- ③ 卡-洛变换.
- ④

The Walsh-Hadamard Transform

- 离散 Walsh 变换

- ① Walsh 变换的核矩阵只包含两个数值: $+1$ 和 -1 , 且变换阵的各行各列之间彼此是正交的.
- ② Walsh 变换与 Hadamard 变换具有相似之处, 两者的区别仅仅在于行的次序不同, 所以常把这一类变换称作 Walsh-Hadamard 变换.
- ③ Walsh-Hadamard 的核是可分离的, 变换阵是实元, 运算只有加法, 简单. 变换有快速算法.
- ④ Walsh-Hadamard 主要用于图像压缩.
- ⑤ Walsh-Hadamard 是非正弦类变换, 运算时不需要乘法. 在六、七十年代曾被推崇; 到八十年代, DSP 芯片的问世, 以及具有优良性能的新变换的提出, 使得正弦类变换无论是在理论价值还是应用价值方面都取代了非正弦类变换, 从而在正交变换中占据了主导地位.

The Walsh-Hadamard Transform

- 离散 Walsh 变换: 一维的形式

- 离散 Walsh 变换要求进行变换的向量, 其长度 N 满足要求: $N = 2^n$, 其中 n 是正整数.
- 一维离散 Walsh 变换:

$$W(u) = \frac{1}{N} \sum_{x=0}^{N-1} f(x) \prod_{i=0}^{n-1} (-1)^{b_i(x)b_{n-1-i}(u)}$$

- 变换核矩阵的元素为:

$$h(x, u) = \frac{1}{N} \prod_{i=0}^{n-1} (-1)^{b_i(x)b_{n-1-i}(u)}$$

- 其中 $b_i(x)$ 表示数 x 的二进制表示的第 i 位, 如 $b_1(4) = 0$, $b_2(4) = 1$.

The Walsh Kernel matrix

- The kernel matrix of DWT

- 设 $N = 8$, 则 $n = 3$. 以 $u = 2, x = 3$, 即 计算 $h(u = 2, x = 3)$ 为例

- ① 此时, u 的二进制表示为 **010**, x 的二进制表示为 **011**. 参加 $\prod_{i=0}^{n-1} (-1)^{b_i(x)b_{n-1-i}(u)}$ 的各数情况如下

i	$b_i(x)$	$b_{n-1-i}(u)$	$(-1)^{b_i(x)b_{n-1-i}(u)}$
0	1	0	1
1	1	1	-1
2	0	0	1

- ② 即 $h(u = 2, x = 3) = -1$

The Walsh Kernel matrix

- The kernel matrix of DWT

- 下图是 $N = 8$ 的 **Walsh** 变换矩阵.

$$W = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & -1 & -1 & -1 & -1 \\ 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 \\ 1 & 1 & -1 & -1 & -1 & -1 & 1 & 1 \\ 1 & -1 & 1 & -1 & 1 & -1 & 1 & -1 \\ 1 & -1 & 1 & -1 & -1 & 1 & -1 & 1 \\ 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 \\ 1 & -1 & -1 & 1 & -1 & 1 & 1 & -1 \end{bmatrix}$$

The Walsh Kernel matrix

- 二维离散 Walsh 正变换

$$W(u, v) = \frac{1}{N} \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) \prod_{i=0}^{n-1} (-1)^{[b_i(x)b_{n-1-i}(u) + b_i(y)b_{n-1-i}(v)]}$$

- 二维离散 Walsh 反变换

$$f(x, y) = \frac{1}{N} \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} W(u, v) \prod_{i=0}^{n-1} (-1)^{[b_i(x)b_{n-1-i}(u) + b_i(y)b_{n-1-i}(v)]}$$

- 注意: 二维离散 Walsh 正变换和反变换的变换核是可分离和对称的.

哈达玛变换, The Hadamard Transform

- Hadamard 变换是一种特殊排列的 Walsh 变换
 - ① Hadamard 变换与 Walsh 变换的区别仅仅在于行的次序不同, 所以常把这一类变换称作 Walsh-Hadamard 变换 .
 - ② 与 Walsh 变换相比, Hadamard 变换的优点在于它的变换核矩阵具有简单的递推关系.

The Walsh-Hadamard Transform

- 离散 Hadamard 变换: 一维的形式

- 离散 Hadamard 变换要求进行变换的向量, 其长度 N 满足要求: $N = 2^n$, 其中 n 是正整数.
- 一维离散 Hadamard 变换:

$$H(u) = \frac{1}{N} \sum_{x=0}^{N-1} f(x) (-1)^{\sum_{i=0}^{n-1} b_i(x)b_i(u)}$$

- 变换核矩阵的元素:

$$h(x, u) = \frac{1}{N} (-1)^{\sum_{i=0}^{n-1} b_i(x)b_i(u)}$$

- 其中 $b_i(x)$ 表示数 x 的二进制表示的第 i 位, 如 $b_1(4) = 0$, $b_2(4) = 1$.

哈达玛变换, The Hadamard Transform

- 最低阶 $N = 2$ 离散 Hadamard 变换矩阵:

$$H_2 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

- $2N$ 阶的 **Hadamard** 矩阵 H_{2N} 与 H_N 之间存在如下的递推关系

$$H_{2N} = \begin{bmatrix} H_N & H_N \\ H_N & -H_N \end{bmatrix}$$

- 即 变换矩阵是一个分块矩阵.
- **Matlab** 可以使用函数 `hadamard(N)` 给出 N 阶的 哈达玛 矩阵

哈达玛变换, The Hadamard Transform

- The kernel matrix of DHT

- 下图是 $N = 8$ 的哈达玛变换矩阵.

$$H_8 = \frac{1}{8} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 & 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 \\ 1 & 1 & 1 & 1 & -1 & -1 & -1 & -1 \\ 1 & -1 & 1 & -1 & -1 & 1 & -1 & 1 \\ 1 & 1 & -1 & -1 & -1 & -1 & 1 & 1 \\ 1 & -1 & -1 & 1 & -1 & 1 & 1 & -1 \end{bmatrix}$$

- 每一行中, 符号变化的次数是不同的. **The sign change count is called sequency(列率).**
- 将哈达玛变换矩阵的行按列率从小到大递增地排列将形成另一种变换: **Ordered Hadamard transform**

哈达玛变换, The Hadamard Transform

- 二维离散 Hadamard 变换:

$$H(u, v) = \frac{1}{N} \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) (-1)^{\sum_{i=0}^{n-1} (b_i(x)b_i(u) + b_i(y)b_i(v))}$$

- 二维离散 Hadamard 反变换:

$$f(x, y) = \frac{1}{N} \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} H(u, v) (-1)^{\sum_{i=0}^{n-1} (b_i(x)b_i(u) + b_i(y)b_i(v))}$$

哈达玛变换, The Hadamard Transform

- 二维离散 Hadamard 变换核是可分离和对称的:

$$h(x, y; u, v) = \left(\frac{1}{\sqrt{N}} (-1)^{\sum_{i=0}^{n-1} b_i(x)b_i(u)} \right) \left(\frac{1}{\sqrt{N}} (-1)^{\sum_{i=0}^{n-1} b_i(y)b_i(v)} \right)$$

- 即: 二维 hadamard 变换可分成两步一维变换完成.